UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

Virtual Coach

Rebecca Micol Finzi
Gianluca Licciardello
Matteo Monacis

Project Overview

The goal of this project is to develop a Virtual Coach system that **assists users in performing gym exercises correctly**, without the need for a human trainer.

The project is divided into three modules:

1) Human Activity Recognition (HAR) – The user's activity is recognised from 1D motion signals (accelerometer + gyroscope), distinguishing dynamic activities and static postures using an ensemble of classical ML and deep learning models.

2) Pose Estimation (Object Detection) – Extracts body keypoints (shoulders, hips, knees, etc.) from each video frame using MediaPipe, providing a lightweight skeletal representation used as input for the exercise recognition and pose-correctness module.

3) Exercise & Pose Quality Recognition – Uses webcam video to recognise which exercise the user is performing and whether the pose is held correctly, driving a timer that advances only while the pose is valid.

01. HAR

Human Activity Recognition

TASK 1

Objectives

Automatically recognize the **physical activity** performed by a person from smartphone accelerometer and gyroscope data.
Task: Classify motion into **six categories**:

- 🚶 **Walking**
- 🔺 **Walking Upstairs**
- 🔻 **Walking Downstairs**
- 🪑 **Sitting**
- 🧍 **Standing**
- 🛌 **Laying**

HAR Using Smartphones Dataset

The **Human Activity Recognition Using Smartphones dataset** was developed by the University of California, Irvine (UCI) and is available in the UCI Machine Learning Repository



Data Collection

- Data collected from **30 volunteers** (aged 19–48)
- Participants wore a Samsung Galaxy S II **on the waist**
- Activities recor using smartphone sensors



Sensors

- Accelerometer: measures body acceleration along the X, Y, and Z axes
- Gyroscope: measures angular velocity and rotation along the X, Y, and Z axes



Data Types

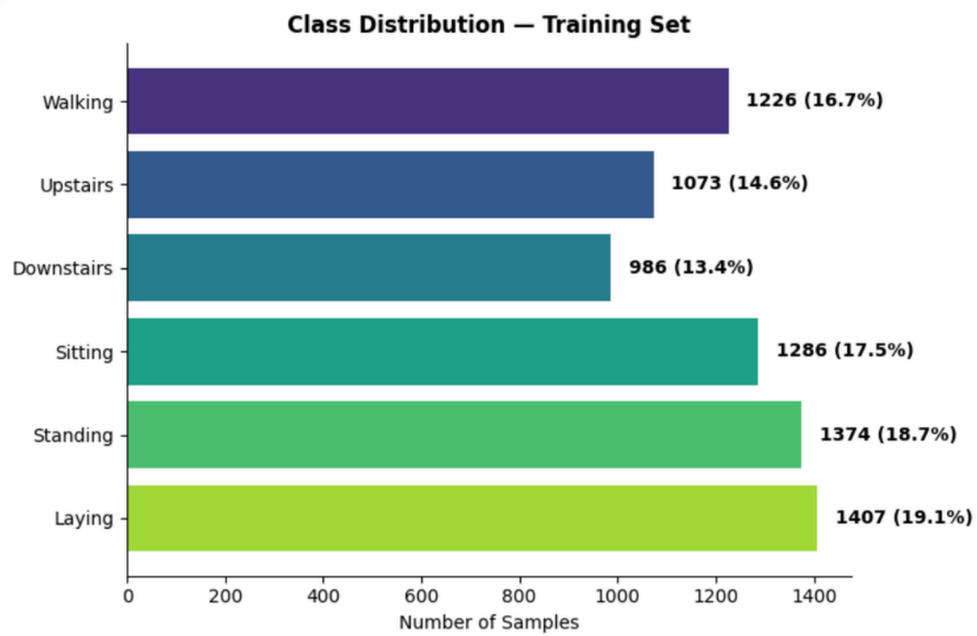
1. **Raw Signals:** direct measurements from accelerometer and gyroscope

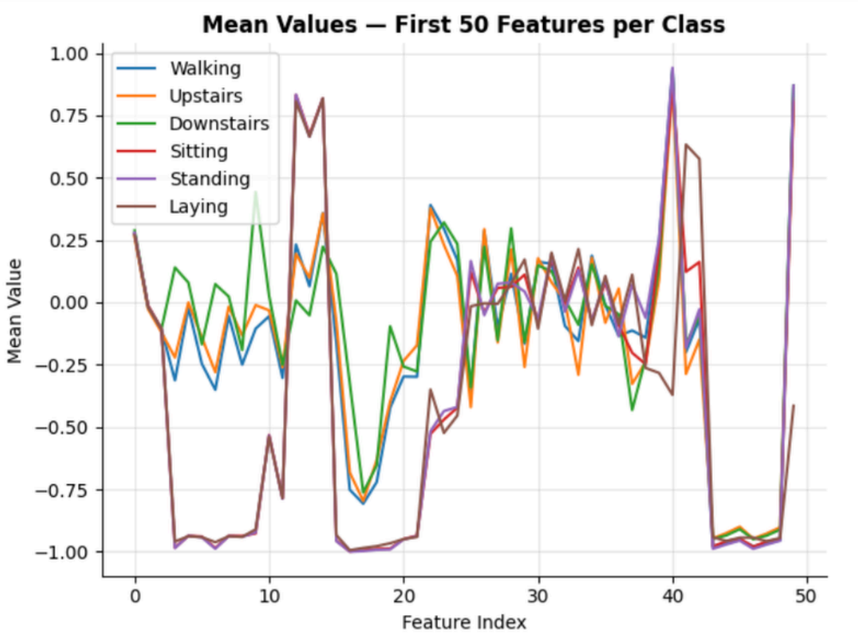


2. **Processed Features:** statistical features extracted from raw signals and used for activity classification
   - 10,299 observations
   - 561 extracted features

   6 activity classes: Walking, Walking Upstairs, Walking Downstairs, Sitting, Standing, and Laying.

Exploratory Data Analysis (EDA)





- The dataset is **well-balanced** (13–19% per class), no strong imbalance issues
- Both **dynamic activities** and **static activities** are quite similar to **each other** within their own group
- **Walking, Upstairs, and Downstairs** have very similar feature patterns (high overlap)
- **Downstairs** is the least frequent and is expected to be the **hardest** to classify

Preprocessing



- **Label Encoding:** Activity labels remapped from 1–6 to 0–5, since most ML/DL models expect classes to start at index 0



- **One-Hot Encoding:** Each label converted into a binary vector (e.g. class 3 → [0,0,1,0,0,0]), required by the softmax output layer of neural networks to compare predicted probabilities against the true class



- **Standard Scaling:** each of the 561 features rescaled to mean = 0, std = 1, since features have very different original ranges and models like SVM and neural networks are sensitive to feature scale → Scaler fitted only on training data, then applied to test data (no data leakage)

Random Forest

Accuracy

~ 92.6 %

Test Set

Input: 561 processed features

Why this model?

- Classic baseline for the UCI HAR dataset on pre-computed features
- **Fast** to train, no heavy hyperparameter tuning needed
- **Interpretable** via feature importance

How it works?

- Ensemble of **100 decision trees**
- Each tree trained on a random subset of features
- Each tree predicts an activity
- Final prediction = class receiving the majority of votes across all trees

Limitation

- Perfect on **"Laying":** easy to separate due to unique gravity orientation
- Main weakness: confuses **Sitting ↔ Standing** (both static, very similar gravity signal from waist) — this is a structural limitation, present in all models
- Some confusion between Downstairs / Walking / Upstairs due to similar rhythmic foot-strike pattern

SVM with RBF kernel

Accuracy

~ 95.49 %

Test Set

Input: 561 processed features

Why this model?

- Second most cited baseline for UCI HAR
- Typically achieves the **highest accuracy** among classical models on this dataset

How it works?

- Searches for the **better boundary** between classes in a **high-dimensional space**
- Maps the **input** into a **high-dimensional space** using a **Radial Basis Function (RBF) kernel**
- Finds the maximum-margin hyperplane that separates the classes
- Hyperparameters: C=10, gamma='scale'
- probability = True enabled to get class probability estimates

Limitation

- Same residual confusion as Random Forest: **Sitting ↔ Standing**
- **More computationally expensive** than Random Forest

Light Gradient Boosting

Accuracy

~ 93.55 %

Test Set

Input: 561 processed features

Why this model?

- Modern, highly efficient gradient boosting algorithm
- Theoretically well-suited for dense, continuous tabular features
- Typically faster to converge than XGBoost with comparable or better accuracy

How it works?

- Gradient boosting model: **builds trees sequentially**, each one correcting the errors of the previous one
- GOSS (Gradient-based One-Side Sampling): keeps the **most informative samples** (large gradients), drops less useful ones → faster training
- EFB (Exclusive Feature Bundling): **groups mutually exclusive features together**, reducing dimensionality
- Key params: n_estimators=500, learning_rate=0.05, num_leaves=63

Limitation

- Slightly **below SVM in accuracy**
- Being sequential (vs Random Forest's parallel bagging), it's more **sensitive to inter-feature dependencies**, but also more prone to overfitting
- Same structural weakness across models: **Sitting ↔ Standing** confusion **persists**

InceptionTime (Multi-Scale CNN)

Accuracy

~ 90.7 %

Test Set

Input: raw signals (128 timesteps × 9 channels)

Why this model?

- State-of-the-art architecture for **time series classification**
- Works directly on **raw sensor signals**, no manual feature engineering needed
- **Captures patterns at multiple time scales simultaneously** (slow postural changes + fast periodic movements)

How it works?

- Convolutional neural network, designed specifically for **signals that change over time**
- Detects **local patterns** within the signal
- Inception module: 3 parallel Conv1D branches with different kernel sizes run at the same time, each capturing a different "speed" of pattern (slow/medium/fast)
- A 1×1 bottleneck convolution reduces dimensionality before the large convolutions, cutting computational cost
- ~234K parameters

Limitation

- Lower accuracy
- Struggles more with "Sitting"

DeepConvLSTM

Accuracy

~ 90.91 %

Test Set

Input: raw signals (128 timesteps × 9 channels)

Why this model?

- Standard deep learning architecture for HAR on raw signals
- Combines two strengths: **CNN** (Convolutional Neural Network) finds local patterns, **LSTM** (Long Short-Term Memory) connects information across the whole sequence
- Fully end-to-end: no manual feature engineering needed, reproducible on any IMU dataset

How it works?

- Conv1D layers: extract local temporal features (acceleration peaks, direction changes, foot-strike patterns) directly from the raw signal
- LSTM layers: model how these local patterns evolve over the full 2.56-second window, capturing temporal dynamics
- ~167K parameters

Limitation

- Sequential LSTM processing is slower to train than fully parallel architectures (e.g. Random Forest, CNN-only)
- Accuracy still below the classical ML baselines (RF, SVM, LightGBM), confirming raw-signal deep learning is a harder task on this dataset size

Transformer with Positional Encoding

Accuracy

~ 88.36 %

Test Set

Input: raw signals (128 timesteps × 9 channels)

Why this model?

- **Modern architecture,** alternative to recurrence (LSTM), **based on self-attention**
- **Captures long-range dependencies** without the vanishing gradient problem
- **Particularly suited to cyclic activities** (walking, stairs) where the start and end of a movement cycle are related

How it works?

- Multi-Head Self-Attention: every timestep can directly "look at" every other timestep in the sequence
- **Positional Encoding:** since attention has **no built-in sense of order,** sinusoidal positional encoding is added **to preserve temporal information**
- Architecture: 2 encoder blocks, d_model=64, ~72K parameters

Limitation

- **Lowest accuracy among all 6 models**
- No warm-up learning rate schedule (used fixed LR=1e-3 with ReduceLROnPlateau); a linear warm-up over the first epochs — standard in the original paper — could improve early training stability
- Self-attention typically needs **more data** to show its full advantage; with a moderate-size dataset like UCI HAR, it **underperforms** compared to CNN

Soft Voting Ensable

Combining six models

Random Forest | SVM (RBF) | Light GBM | Inception Time | Transf. + PE | DeepConv LSTM

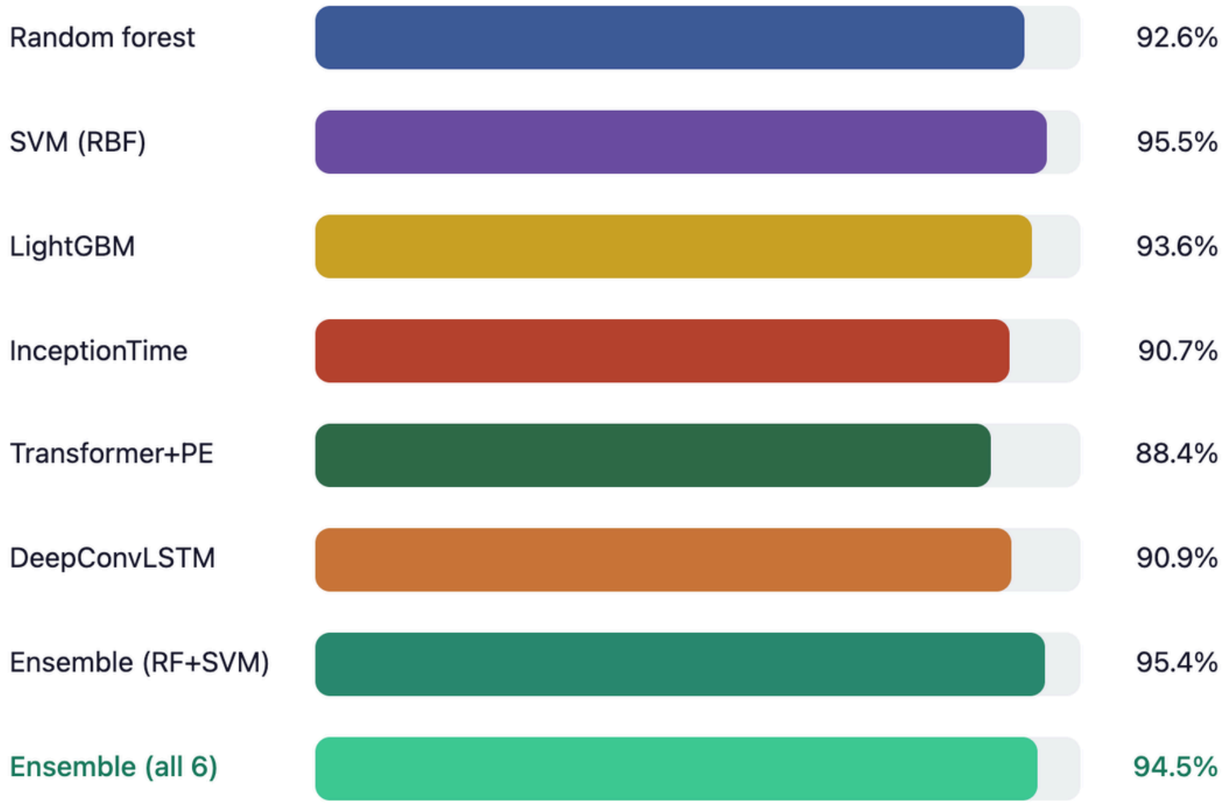
Why this model?

- **Combines strengths** of both classical ML (RF, SVM, LightGBM) and deep learning (InceptionTime, Transformer, DeepConvLSTM)

How it works?

- **Soft voting:** each model outputs a probability distribution over classes; these are averaged (equal weight, 1/6 each)
- **Final prediction** = class with the highest average probability
- **Uniform weights** used (not accuracy-based) to avoid data leakage from the test set

Conclusion



- The **full 6-model ensemble (94.5%) does not beat SVM**
- **Best single model: Support Vector Machine with an Radial Basis Function kernel** at 95.5%
- Classical ML (RF, SVM, LightGBM) on engineered features **beats deep learning on raw signals** (InceptionTime, Transformer, DeepConvLSTM)
- Structural limitation across all models: confusing Sitting vs Standing, due to the physical similarity of these postures from a waist-mounted sensor

02. Pose Estimation

TASK 2

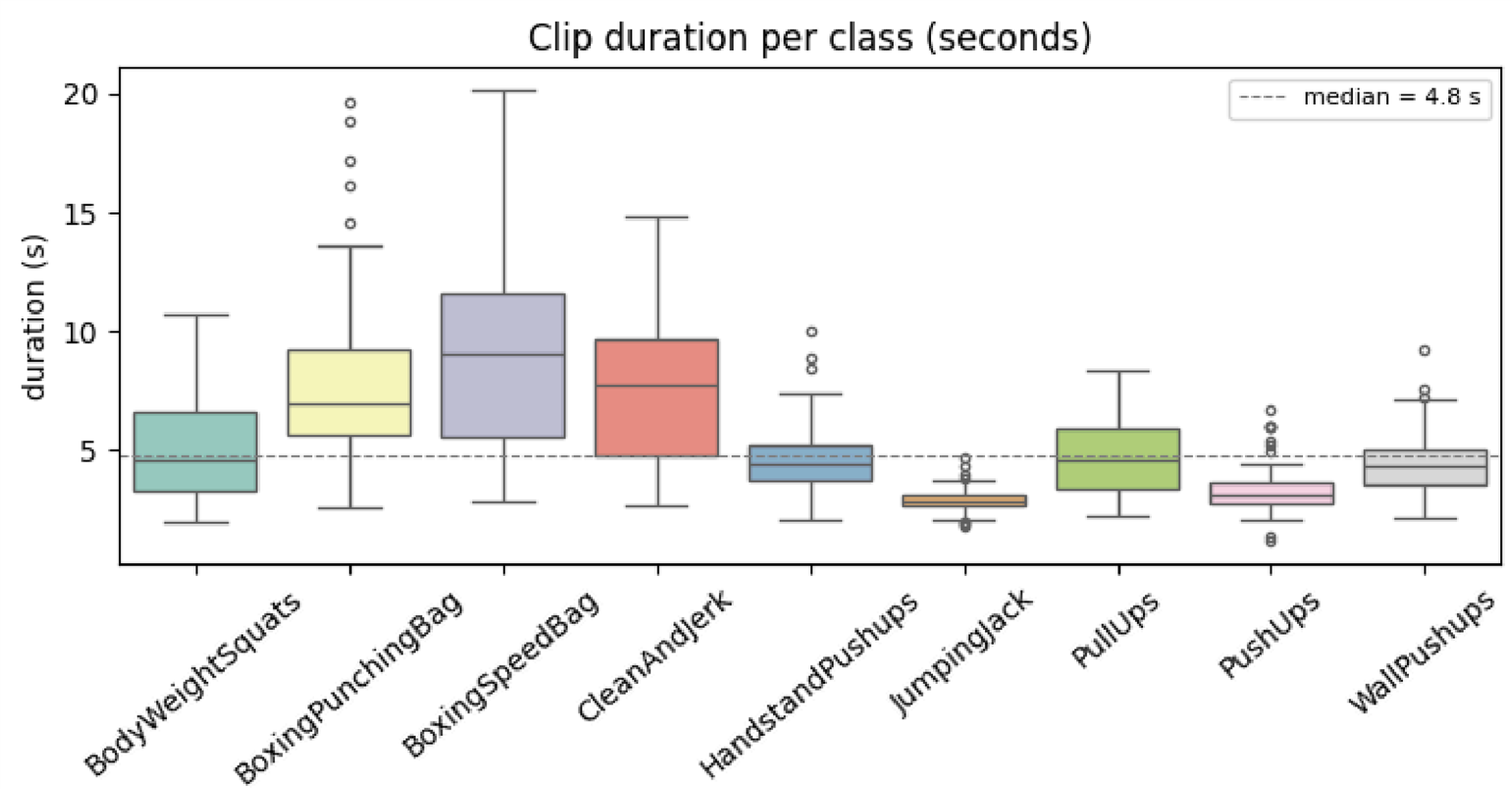
UCI-101

The UCI-101 dataset is a collection of videos containing 101 different human action classes (LINK),

We selected 9 exercises according to the aim of the project and reproducibility.

- Body Weight Squats (112 clips)
- Boxing Punching Bag (163 clips)
- Boxing Speed Bag (134 clips)
- Clean and Jerk (112 clips)
- Handstand Push Ups (128 clips)
- Jumping Jack (123 clips)
- Pull Ups (100 clips)
- Push Ups (102 clips)
- Wall Push Ups (130 clips)

UCI-101



- Videos are in a range of 25-30 FPS
- Boxing Speed Bag, Boxing Punching Bag and Clean And Jerk classes have clearly longer videos
- The median video length is lower than 5 seconds, this justifies our choice to do not cut videos in shorter clips
- The downside of this choice is represented by the short video duration of Jumping Jack and Push Ups classes

MediaPipe - BlazePose



33 body keypoints:

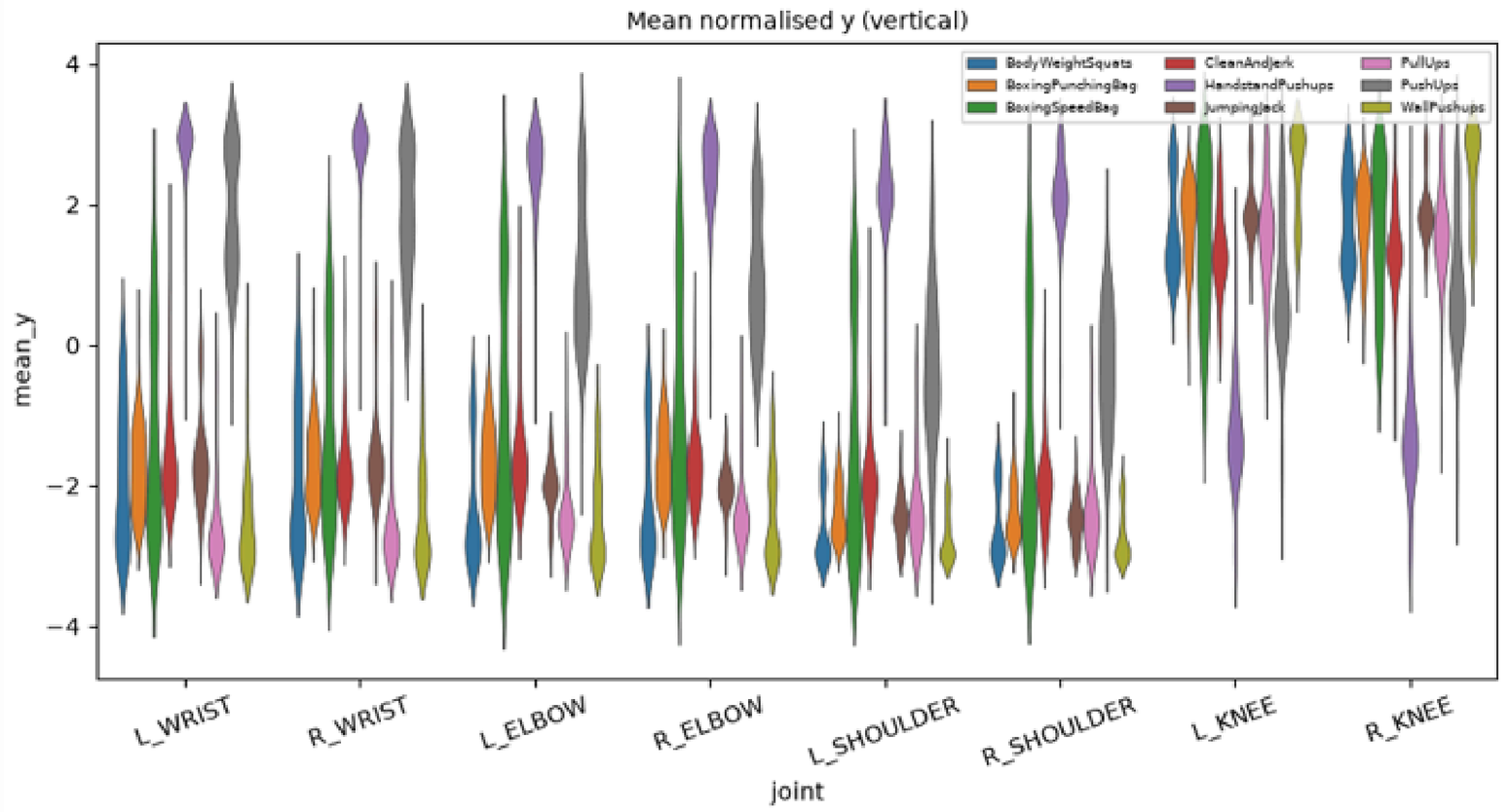
- 3 for each feet
- 1 for each knee
- 1 for each hip
- 1 for each shoulder
- 3 for each hand
- 1 for each wrist
- 1 for each elbow
- 2 for the mouth
- 1 for the nose
- 3 for each each
- 1 for each ear

Input: 256x256x3 frames

Output: (x, y, z, visibility, presence) where:

- **Visibility** is the probability that a keypoint is located within the frame and not occluded.
- **Presence** is the probability that a keypoint is located within the frame.

Comparison



- Y is the most discriminative coordinate
- Finger, toe and head keypoints should not be as much informative as leg and arm ones
- Double-bump and longer shapes capture movement, such as all arms keypoints for Boxing Speed Bag
- Single-bumb and shorter shapes highlight static body parts. For example, shoulders in Wall Push Ups

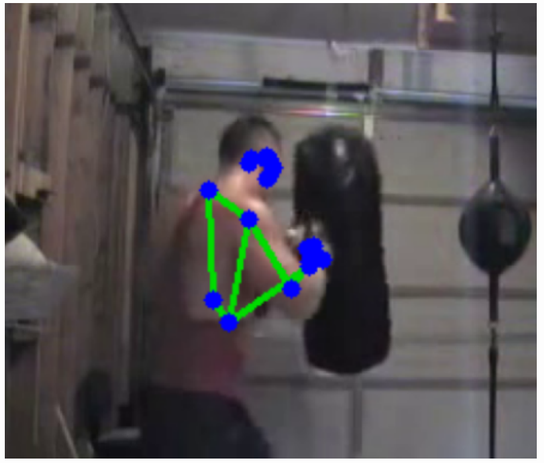
Comparison

Overall visual performance of MediaPipe is good: skeleton are clearly recognisable, legs and arms keypoints are accurate (cannot confirm for face, hands and feet ones due to distance reasons)

Issues:

- Body parts might be not be in the framing or partially occluded
- Estimations may be also completely wrong due to MediaPipe limitations
- There might be more people within the frame, MediaPipe is unable to recognise more people at time within the same frame.
- Sometimes MediaPipe does not detect the person maybe for lighting condition issues or others. Bottom left images shows a man with a white shirt sorrounded by white items









03.

Exercise Recognition, Repetition Counting & Posture Assessment

TASK 3

Objectives

From the Task 3 skeleton keypoints: recognise which of 9 exercises is performed, count repetitions in real time, and assess posture — driving a timer that advances only while the exercise is executed correctly.

- A — Exercise Recogniser (deep-learning classifier)
- B — Repetition Counter (adaptive finite-state machine)
- C — Posture Assessment (per-rep quality checks)

Recogniser Screening — Setup & Results

Identical setup for all models

screen_fit() is called identically for every model in run_screening() — parameters hardcoded:

Optimizer	`AdamW (lr=1e-3, wd=1e-4)`
Epochs	`SCREEN_EPOCHS = 25`
Batch size	`128 GPU / 64 CPU`
Loss	`cross_entropy`
AMP fp16	`AMP_ENABLED (all)`
Class balance	`weighted sampling`
Model selection	`best val accuracy`
Seed	`manual_seed(42), reset`

Screening 2D (input x, y)

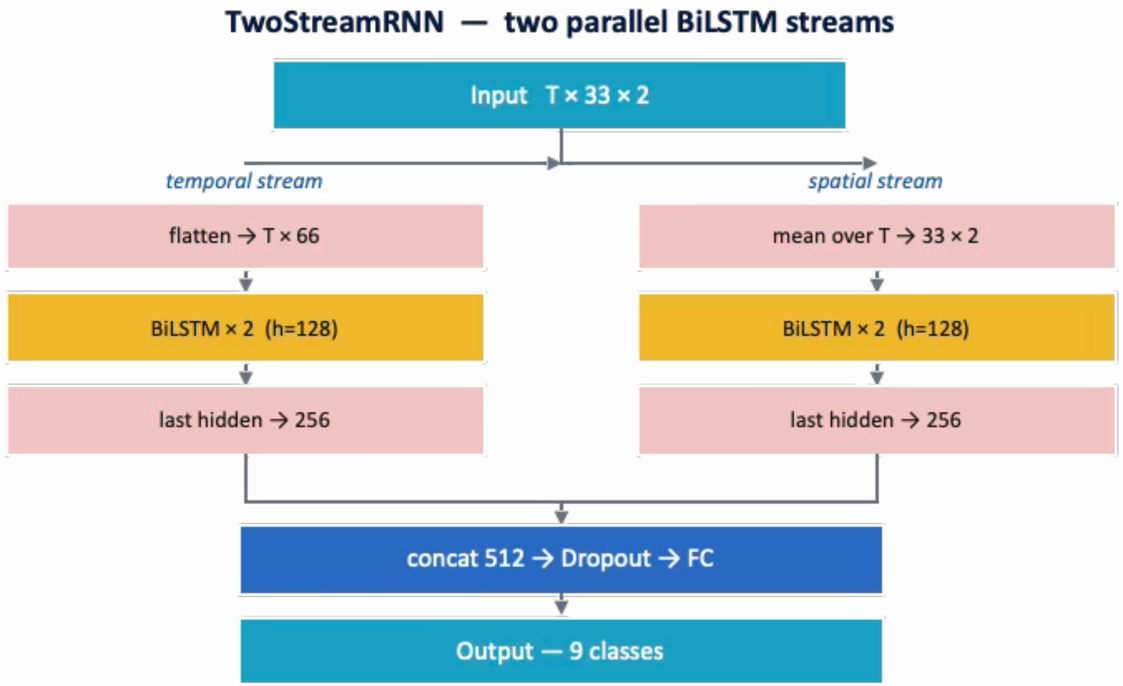
Rank	Model	Val Acc	Test Acc	Params (k)	Time (s)
1	**TwoStreamRNN**	**0.9191**	0.9504	1131.0	2.2
2	JTMNet	0.9191	0.9433	94.6	10.0
3	MultiOrderTransformer	0.9044	0.9149	536.8	23.9
4	CTR-GCN	0.8382	0.8511	97.5	11.0

Screening 3D (input x, y, z)

Rank	Model	Val Acc	Test Acc	Params (k)	Time (s)
1	**PartLSTM**	**0.9044**	0.8936	108.1	1.9
2	TwoStreamAGCN	0.8824	0.8723	125.3	13.6
3	SkeleTR	0.8309	0.8652	275.1	2.0
4	EDM-3DCNN	0.5000	0.4681	14.7	8.3

Winners → TwoStreamRNN (2D) and PartLSTM (3D) advance to grid search + 5-fold CV.

Grid Search & Winning Architectures



Full training setup — grid-search phase

Epochs	`FULL_EPOCHS = 60` more than doubled vs screening
Augmentation	`flip · ±15° rot · scale 0.8–1.2 · jitter` applied every sample, every epoch
Early stopping	`patience=10, min 15 ep` best state on val LOSS
CV split	`StratifiedKFold(k=5, seed=42)` train+val pooled, per-fold split
Selection	`cv_mean` mean val accuracy over 5 folds

Unchanged: AdamW · cross_entropy · WeightedRandomSampler · AMP · batch=128

TwoStreamRNN *(2D winner)*

`hidden` 96 or 128
(both LSTM streams)

`dropout` 0.3 or 0.5

`lr` fixed 1e-3 (not searched)

2 × 2 × 1 = 4 combos × 5-fold → 20 trainings

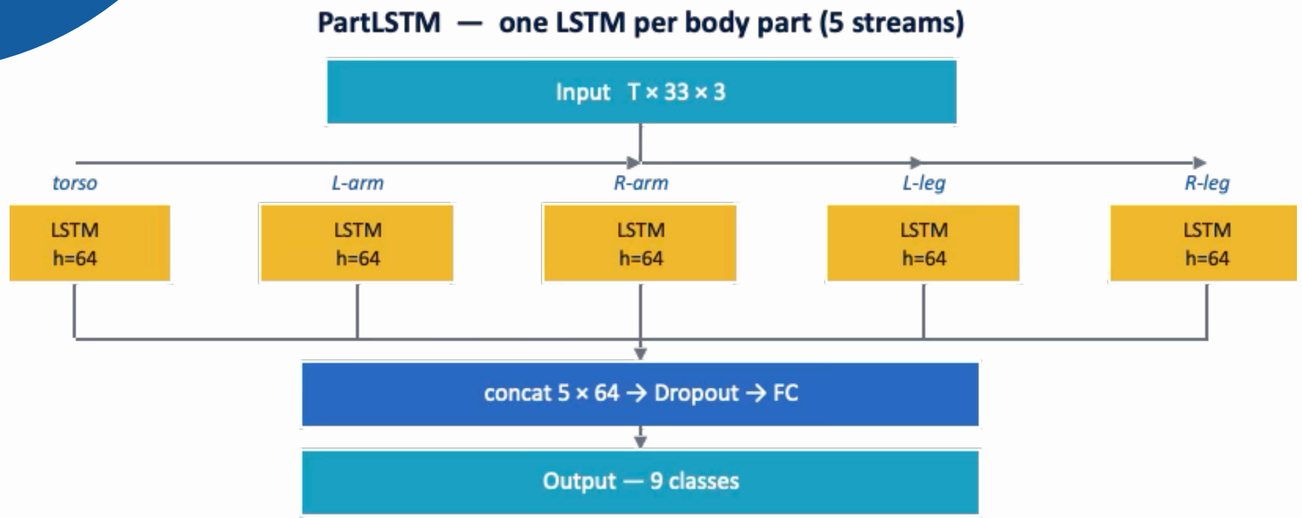
Best config
`hidden=128, dropout=0.3, lr=1e-3`

TwoStreamRNN *(2D — best overall)*

Screening Test	CV Mean ± Std	Final Refit Test	Macro F1
0.9504	0.9361 ± 0.0164	0.9433	**0.936**

Exercise	Precision	Recall	F1	Support
BodyWeightSquats	0.933	1.000	0.966	14
BoxingPunchingBag	1.000	1.000	1.000	21
BoxingSpeedBag	1.000	0.941	0.970	17
CleanAndJerk	0.917	0.786	0.846	14
HandstandPushups	1.000	1.000	1.000	16
JumpingJack	1.000	1.000	1.000	16
PullUps	0.846	0.846	0.846	13
PushUps	0.750	0.923	0.828	13
WallPushups	1.000	0.941	0.970	17

Grid Search & Winning Architectures



PartLSTM *(3D winner)*

`hidden` 48 or 64
(each of the 5 part-LSTMs)

`dropout` 0.3 or 0.5

`lr` fixed 1e-3 (not searched)

2 × 2 × 1 = 4 combos × 5-fold → 20 trainings

Best config
`hidden=64, dropout=0.3, lr=1e-3`

PartLSTM *(3D)*

Screening Test	CV Mean ± Std	Final Refit Test	Macro F1
0.8936	0.9204 ± 0.0121	0.9220	**0.916**

Exercise	Precision	Recall	F1	Support
BodyWeightSquats	0.875	1.000	0.933	14
BoxingPunchingBag	1.000	0.905	0.950	21
BoxingSpeedBag	1.000	0.882	0.938	17
CleanAndJerk	0.833	0.714	0.769	14
HandstandPushups	1.000	1.000	1.000	16
JumpingJack	1.000	0.938	0.968	16
PullUps	0.917	0.846	0.880	13
PushUps	0.722	1.000	0.839	13
WallPushups	0.944	1.000	0.971	17

Online Repetition Counter - Adaptive FSM

The problem

Fixed thresholds from the literature (e.g. 130°/150° elbow angle) break when camera angle or distance changes.

The signal's operating range shifts — but the thresholds don't.

The solution: adapt online to the user's motion

- One scalar per frame: a joint angle (squats, push-ups, pull-ups, clean-and-jerk, jumping jacks)or a wrist-extension distance (boxing)
- A sliding window tracks the live min / max of that feature
- **Thresholds set at 35% (low) and 65% (high) of the live range — never absolute**
- The counter uses **two separate thresholds instead of one**: the signal must drop below the low threshold (effort phase) and then rise back above the high threshold (return to rest) to count one rep — using two thresholds instead of one prevents small oscillations from triggering false counts
- Refractory period (5 frames) + minimum-range gate reject physically impossible reps
- The same signal range is also used to flag shallow reps and data-quality issues

No absolute thresholds — everything is relative to the single clip's own signal range.

Posture Assessment & Session Integration

Part C — Quality checks (non-corrective)

- **Per-rep range of motion, reported as full / shallow / insufficient data**
- check_pose_dropout — fraction of frames where key joints fell below the visibility threshold
- check_signal_range_sufficiency — did the feature move enough to represent real reps?
- check_anatomical_plausibility — flags a signal stuck in an implausible zone
- Every count is explained — a zero count is never silent

Part D — VirtualCoachSession

- **User sets: exercise, reps per set, number of sets**
- Per frame: normalise → recognise → count → assess form
- **Recognition gating: the counter advances only when the recogniser matches the chosen exercise (≥ 50% confidence)**
- EMA smoothing stabilises the frame-by-frame prediction
- Automatic set tracking and workout-completion signalling

Thank you !